

Paso 3 - Variables y operaciones | Solucionario

Las seis soluciones del paso 3. Todas son programas sin teclado: los datos están escritos en el código, así que tu salida tiene que dar exactamente estos números.

Úsalo después de intentarlo. Cada solución viene con la salida real del programa: si la tuya coincide, está bien aunque el código sea distinto. Los `.java` de esta carpeta son los mismos que se muestran abajo, listos para ejecutar: `cd soluciones/paso03` y `javac *.java`.

B3.1 - El IVA

Tres valores que salen de **una** operación: el IVA se calcula, y el total se calcula sumando, no escribiendo 11900 a mano.

▼ Ver la solución (B3_1_Iva.java)

B3_1_Iva.java

```
public class B3_1_Iva {
    public static void main(String[] args) {
        int neto = 10000;
        final double IVA = 0.19;

        double iva = neto * IVA;
        double total = neto + iva;

        System.out.printf("Neto: %5.0f%n", (double) neto);
        System.out.printf("IVA: %5.0f%n", iva);
        System.out.printf("Total: %5.0f%n", total);
    }
}
```

Salida:

```
Neto:  10000
IVA:   1900
Total: 11900
```

Lo que importa. Que `iva` y `total` sean **variables calculadas**. Si mañana el IVA sube al 20%, se cambia **una** línea y los tres números quedan bien; si los escribiste a mano, hay que recalcular todo.

El `%5.0f` imprime sin decimales y reserva 5 espacios, así los tres números quedan alineados por la derecha. Con `(int) iva` también sirve, pero ojo: el cast **corta**, no redondea, y `%.0f` sí redondea. Con 1900.0 da lo mismo; con 1900.6 no.

B3.2 - La película

La división entera da las horas y el resto (`%`) da los minutos. Hay que repetir el cálculo para las tres duraciones.

▼ Ver la solución (B3_2_Pelicula.java)

B3_2_Pelicula.java

```
public class B3_2_Pelicula {
    public static void main(String[] args) {
        int duracion = 135;
        System.out.println("Dura " + duracion / 60 + " horas y " + duracion %
60 + " minutos");

        duracion = 200;
        System.out.println("Dura " + duracion / 60 + " horas y " + duracion %
60 + " minutos");

        duracion = 59;
        System.out.println("Dura " + duracion / 60 + " horas y " + duracion %
60 + " minutos");
    }
}
```

Salida:

```
Dura 2 horas y 15 minutos
Dura 3 horas y 20 minutos
Dura 0 horas y 59 minutos
```

Lo que importa. `135 / 60` da **2** y no 2.25 justamente porque los dos son enteros: aquí la división entera, que en el paso 3 era una trampa, es exactamente lo que necesitas. Y `135 % 60` da 15, los

minutos que sobran. Las dos operaciones van siempre juntas cuando conviertes unidades.

Fíjate en lo fea que quedó la repetición: la misma línea escrita tres veces, cambiando un número. En el **paso 9** eso se convierte en `static void mostrarDuracion(int minutos)` y se llama tres veces. Guarda este programa para compararlo entonces.

B3.3 - Los tres dígitos

`% 10` saca el último dígito y `/ 10` lo elimina. Repitiendo eso tres veces salen las unidades, las decenas y las centenas.

▼ Ver la solución (B3_3_Digitos.java)

B3_3_Digitos.java

```
public class B3_3_Digitos {
    public static void main(String[] args) {
        int numero = 573;

        int unidades = numero % 10;           // 3
        int decenas = (numero / 10) % 10;      // 57 -> 7
        int centenas = numero / 100;          // 5

        System.out.println("Centenas: " + centenas);
        System.out.println("Decenas: " + decenas);
        System.out.println("Unidades: " + unidades);
        System.out.println("Suma de los digitos: " + (centenas + decenas +
unidades));
    }
}
```

Salida:

```
Centenas: 5
Decenas: 7
Unidades: 3
Suma de los digitos: 15
```

Lo que importa. La pareja `/ 10` y `% 10` es **la** herramienta para trabajar con los dígitos de un número, y vuelve en el B6.5 (dar vuelta un número) y en el B11.4 (el dígito verificador del RUT).

Conviene entenderla ahora:

- `% 10` → el último dígito.
- `/ 10` → el número **sin** el último dígito.

Ojo con los paréntesis de `(numero / 10) % 10`. Sin ellos, `numero / 10 % 10` da lo mismo (los dos operadores tienen la misma precedencia y se leen de izquierda a derecha), pero se entiende mucho peor. Los paréntesis no son para Java, son para el que lee.

B3.4 - El promedio mal calculado

El error es la división entera: `(5 + 6 + 5) / 3` son **tres enteros**, así que Java divide 16 entre 3 y da 5, y recién después lo guarda como 5.0. Las dos formas de arreglarlo están abajo.

▼ Ver la solución (B3_4_PromedioMalo.java)

B3_4_PromedioMalo.java

```
public class B3_4_PromedioMalo {
    public static void main(String[] args) {
        int nota1 = 5;
        int nota2 = 6;
        int nota3 = 5;

        // MAL: los tres son int, así que la división es entera
        double malo = (nota1 + nota2 + nota3) / 3;
        System.out.println("Mal:          " + malo);

        // Arreglo 1: convertir la suma a double ANTES de dividir
        double bueno1 = (double) (nota1 + nota2 + nota3) / 3;
        System.out.println("Con cast:      " + bueno1);

        // Arreglo 2: que el divisor sea decimal, y listo
        double bueno2 = (nota1 + nota2 + nota3) / 3.0;
        System.out.println("Con el 3.0:    " + bueno2);
    }
}
```

Salida:

```
Mal:           5.0
Con cast:      5.333333333333333
Con el 3.0:    5.333333333333333
```

Las dos formas, y cuál usar. El `3.0` es más corto y más legible cuando el divisor es un número escrito en el código. El `(double)` es obligatorio cuando el divisor es **otra variable** `int`, que es el caso de verdad: `(double) suma / cantidad`.

Y la respuesta del punto 3: con las notas 5, 6 y 4 el promedio real es exactamente 5.0, así que la división entera daba **el mismo resultado por casualidad**. El error estaba ahí desde el principio, pero los datos lo tapaban. Es el tipo de bug que llega a producción: no aparece hasta que alguien pone una nota distinta.

Por eso, cuando pruebes algo, elige datos que **no** sean redondos.

B3.5 - Intercambiar dos variables

Con una tercera variable que guarde el valor que está por perderse.

▼ Ver la solución (B3_5_Intercambio.java)

B3_5_Intercambio.java

```
public class B3_5_Intercambio {
    public static void main(String[] args) {
        int a = 3;
        int b = 8;
        System.out.println("Antes:    a = " + a + ", b = " + b);

        int auxiliar = a;    // guardo el 3 antes de pisarlo
        a = b;                // a vale 8
        b = auxiliar;        // b vale el 3 que tenia guardado

        System.out.println("Despues:  a = " + a + ", b = " + b);
    }
}
```

Salida:

Antes: $a = 3, b = 8$

Despues: $a = 8, b = 3$

Por qué no sirve $a = b; b = a;$. La primera línea **pisa** el valor de a : el 3 se perdió para siempre. En la segunda, a ya vale 8, así que b también queda en 8 y terminas con $a = 8, b = 8$.
Pruébalo: ver el 8 duplicado explica el concepto mejor que cualquier dibujo.

Una asignación no *relaciona* dos variables, **copia** un valor de una a otra. Esa idea es la que se pone interesante en la demo 1, cuando lo que se copia es la referencia a un objeto y los dos nombres terminan apuntando al **mismo** objeto.

B3.6 - Desafío: la calculadora de propinas

El cálculo es directo; lo interesante es el reparto cuando no da exacto.

▼ Ver la solución (B3_6_Propinas.java)

B3_6_Propinas.java

```
public class B3_6_Propinas {
    public static void main(String[] args) {
        double cuenta = 23400;
        int personas = 3;
        final double PROPINA = 0.10;

        double propina = cuenta * PROPINA;
        double total = cuenta + propina;

        System.out.printf("Cuenta:                %6.0f%n", cuenta);
        System.out.printf("Propina (10%%):        %6.0f%n", propina);
        System.out.printf("Total:                %6.0f%n", total);
        System.out.printf("Cada uno paga:        %6.0f%n", total / personas);

        System.out.println();

        // Con 7 personas la division no es exacta
        personas = 7;
        double porPersona = total / personas;
        System.out.println("Con 7 personas, exacto: " + porPersona);
        System.out.printf("Redondeando hacia arriba (ceil):  %.0f%n",
Math.ceil(porPersona));
        System.out.printf("Redondeando al mas cercano:      %.0f%n", (double)
Math.round(porPersona));
        System.out.printf("Con ceil, se juntan %.0f y sobran %.0f pesos%n",
            Math.ceil(porPersona) * personas, Math.ceil(porPersona) *
personas - total);
    }
}
```

Salida:

```
Cuenta:          23400
Propina (10%):   2340
Total:           25740
Cada uno paga:   8580
```

```
Con 7 personas, exacto: 3677.1428571428573
Redondeando hacia arriba (ceil): 3678
Redondeando al mas cercano:      3677
Con ceil, se juntan 25746 y sobran 6 pesos
```

La justificación que pedía el ejercicio. Con `Math.ceil` cada uno paga 3678 y se juntan 25746: sobran 6 pesos. Con `Math.round` cada uno paga 3677 y se juntan 25739: **falta 1 peso** para pagar la cuenta. Repartiendo plata, redondear hacia arriba es lo correcto: es preferible que sobre a que el mozo reciba menos.

Fíjate también en el `%%` del `printf`: para imprimir un `%` literal hay que escribir dos, igual que con la barra invertida del B2.4. Si escribes uno solo, Java cree que empieza un hueco y revienta con `UnknownFormatConversionException`.

Volver a la [hoja del paso 3](#) o al [paso 3 de la guía](#).